Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчет

по Лабораторной Работе № 4

по дисциплине «**Инструментальные средства разработки ПО**»

Автор: Капланян Арман Ованесович
Факультет: ИТиП
Группа: М3102



Задание лабораторной работы

Написать unit-тесты на unittest для функций площади и периметра четырёх фигур (квадрат, прямоугольник, треугольник, круг), не изменяя исходный код функций. Тесты выложить на GitHub, в отчёте составить план тестирования и описать результаты.

Репозиторий: https://github.com/KaplanianArman/Lab4_-

План тестирования

1. Цели и задачи тестирования

Цель — проверить корректность работы функций area и perimeter для всех четырёх фигур при обычных и граничных входных данных.

Задачи: подобрать тестовые наборы данных, заранее рассчитать ожидаемые значения, реализовать автоматические тесты на unittest, выполнить их и проанализировать результаты.

2. Описание тестируемого продукта

Четыре Python-модуля, в каждом — функции area и perimeter:

– square.py: $S = a^2$, $P = 4a$;
– rectangle.py: $S = a \cdot b$, $P = 2(a + b)$;
– triangle.py: $S = a \cdot h / 2$, $P = a + b + c$;
– circle.py: $S = \pi r^2$, $L = 2\pi r$.

Функции принимают целые положительные числа; результат для квадрата и прямоугольника целый, для треугольника и круга — вещественный.

3. Область тестирования

Тестируются только функции area и perimeter всех четырёх модулей. Интерфейс, ввод данных и производительность не проверяются. Код функций не изменяется.

4. Стратегия тестирования

Функциональное модульное тестирование методом «чёрного ящика». Тесты каждой фигуры оформлены отдельным классом-наследником unittest.TestCase.

Позитивные тесты проверяют обычные значения, нулевой и единичный размер, очень большие числа, а для прямоугольника — равные стороны. Целые результаты сравниваются через assertEqual, вещественные — через assertAlmostEqual (с учётом погрешности вычислений).

Негативные тесты проверяют реакцию на некорректные данные: отрицательные числа и строки. По документации функции принимают только целые положительные числа, поэтому в таких случаях ожидается ошибка: ValueError для отрицательных чисел и TypeError для строк. Это проверяется через assertRaises. Запуск:

```
python3 -m unittest rectangle.py square.py triangle.py circle.py
```

5. Критерии приемки

Покрыты все 4 фигуры и обе функции; все 36 тестов завершаются со статусом OK; тесты загружены на GitHub.

6. Ожидаемые результаты

Всего 36 тестов: 20 позитивных и 16 негативных (по 4 на фигуру); S — площадь, P — периметр, L — длина окружности. Ожидается, что все тесты пройдут успешно.

Позитивные тесты

Фигура	Тест	Входные данные	Ожидаемый результат
Квадрат	test_zero_len	a = 0	S = 0, P = 0
	test_default	a = 12	S = 144, P = 48
	test_big_num	a ≈ $8{,}3 \cdot 10^{22}$	точные большие целые
	test_unit_len	a = 1	S = 1, P = 4
Прямоугольник	test_zero_width	a = 0, b = 10	S = 0, P = 20
	test_zero_height	a = 20, b = 0	S = 0, P = 40
	test_square	a = 10, b = 10	S = 100, P = 40
	test_big_num	a = 1190299, b = 19029109	S = 22650329413591, P = 40438816
	test_unit_width	a = 1, b = 902181	S = 902181, P = 1804364
	test_unit_height	a = 9219289832, b = 1	S = 9219289832, P = 18438579666
Треугольник	test_zero_base	a = 0, h = 4; стороны 0, 2, 3	S = 0.0, P = 5
	test_zero_height	a = 1, h = 0; стороны 1, 2, 3	S = 0.0, P = 6
	test_default	a = 6, h = 2; стороны 6, 4, 5	S = 6.0, P = 15
	test_big_num	большие a, h и стороны	S = 2788480570897692.0, P = 19222227412
	test_unit_base	a = 1, h = 6; стороны 1, 3, 3	S = 3.0, P = 7
	test_unit_height	a = 2, h = 1; стороны 2, 4, 5	S = 1.0, P = 11
Круг	test_zero_radius	r = 0	S = 0.0, L = 0.0
	test_default	r = 24	S ≈ 1809.557, L ≈ 150.796
	test_big_num	r = 82398287238719	S ≈ $2.133 \cdot 10^{28}$, L ≈ $5.177 \cdot 10^{14}$
	test_unit_radius	r = 1	S = π, L = 2π

Негативные тесты

Фигура	Отрицательные числа (ожидается ValueError)	Строки (ожидается TypeError)
Квадрат	a = −5; a ≈ $-8{,}3 \cdot 10^{22}$	a = '12'; a = 'abc'
Прямоугольник	a = −10, b = 20; a = −10, b = −20	a = '10', b = 20; a = 'abc', b = 'def'
Треугольник	основание −6; высота −2 и сторона −4	основание и сторона '6'; все аргументы — строки
Круг	r = −24; r = −1	r = '24'; r = 'abc'

7. Актуальные результаты тестирования

Ниже приведён код тестов для каждой фигуры и результат их запуска.

square.py > SquareTestCase > test_zero_len

```python
class SquareTestCase(unittest.TestCase):
    def test_zero_len(self):
        a = 0
        res_area = area(a)
        res_perimeter = perimeter(a)
        self.assertEqual(res_area, 0)
        self.assertEqual(res_perimeter, 0)

    def test_default(self):
        a = 12
        res_area = area(a)
        res_perimeter = perimeter(a)
        self.assertEqual(res_area, 144)
        self.assertEqual(res_perimeter, 48)

    def test_big_num(self):
        a = 8321937812738917382719 8
        res_area = area(a)
        res_perimeter = perimeter(a)
        self.assertEqual(res_area, 692546489590937963589459569406406793876453120 4)
        self.assertEqual(res_perimeter, 3328775125095566953087 92)

    def test_unit_len(self):
        a = 1
        res_area = area(a)
        res_perimeter = perimeter(a)
        self.assertEqual(res_area, 1)
        self.assertEqual(res_perimeter, 4)
```

square.py

rectangle.py > RectangleTestCase > test_unit_height

```python
class RectangleTestCase(unittest.TestCase):
    def test_zero_width(self):
        w, h = 0, 10
        res_area = area(w, h)
        res_perimeter = perimeter(w, h)
        self.assertEqual(res_area, 0)
        self.assertEqual(res_perimeter, 20)

    def test_zero_height(self):
        w, h = 20, 0
        res_area = area(w, h)
        res_perimeter = perimeter(w, h)
        self.assertEqual(res_area, 0)
        self.assertEqual(res_perimeter, 40)

    def test_square(self):
        w, h = 10, 10
        res_area = area(w, h)
        res_perimeter = perimeter(w, h)
        self.assertEqual(res_area, 100)
        self.assertEqual(res_perimeter, 40)

    def test_big_num(self):
        w, h = 1190299, 19029109
        res_area = area(w, h)
        res_perimeter = perimeter(w, h)
        self.assertEqual(res_area, 22650329413591)
        self.assertEqual(res_perimeter, 40438816)

    def test_unit_width(self):
        w, h = 1, 902181
        res_area = area(w, h)
        res_perimeter = perimeter(w, h)
        self.assertEqual(res_area, 902181)
        self.assertEqual(res_perimeter, 1804364)
```

rectangle.py

triangle.py > TriangleTestCase > test_unit_height

```python
class TriangleTestCase(unittest.TestCase):
    def test_zero_base(self):
        a, b, c, h = 0, 2, 3, 4
        res_area = area(a, h)
        res_perimeter = perimeter(a, b, c)
        self.assertAlmostEqual(res_area, 0.0)
        self.assertEqual(res_perimeter, 5)

    def test_zero_height(self):
        a, b, c, h = 1, 2, 3, 0
        res_area = area(a, h)
        res_perimeter = perimeter(a, b, c)
        self.assertAlmostEqual(res_area, 0.0)
        self.assertEqual(res_perimeter, 6)

    def test_default(self):
        a, b, c, h = 6, 4, 5, 2
        res_area = area(a, h)
        res_perimeter = perimeter(a, b, c)
        self.assertAlmostEqual(res_area, 6.0)
        self.assertEqual(res_perimeter, 15)

    def test_big_num(self):
        a, b, c, h = 19209102, 1098118, 19201920192, 290329092
        res_area = area(a, h)
        res_perimeter = perimeter(a, b, c)
        self.assertAlmostEqual(res_area, 2788480570897692.0)
        self.assertEqual(res_perimeter, 19222227412)

    def test_unit_base(self):
        a, b, c, h = 1, 3, 3, 6
        res_area = area(a, h)
        res_perimeter = perimeter(a, b, c)
        self.assertAlmostEqual(res_area, 3.0)
        self.assertEqual(res_perimeter, 7)
```

triangle.py

circle.py > CircleTestCase > test_unit_radius

```python
class CircleTestCase(unittest.TestCase):
    def test_zero_radius(self):
        r = 0
        res_area = area(r)
        res_perimeter = perimeter(r)
        self.assertAlmostEqual(res_area, 0.0)
        self.assertAlmostEqual(res_perimeter, 0.0)

    def test_default(self):
        r = 24
        res_area = area(r)
        res_perimeter = perimeter(r)
        self.assertAlmostEqual(res_area, 1809.5573684677208)
        self.assertAlmostEqual(res_perimeter, 150.79644737231007)

    def test_big_num(self):
        r = 82398287238719
        res_area = area(r)
        res_perimeter = perimeter(r)
        self.assertAlmostEqual(res_area, 2.132977338930098e+28)
        self.assertAlmostEqual(res_perimeter, 517723707715082.44)

    def test_unit_radius(self):
        r = 1
        res_area = area(r)
        res_perimeter = perimeter(r)
        self.assertAlmostEqual(res_area, 3.141592653589793)
        self.assertAlmostEqual(res_perimeter, 6.283185307179586)
```

circle.py



Результат запуска тестов (конец вывода)

Запущено 36 тестов: 23 пройдены, 13 провалены (FAILED, failures=13).

Количество пройденных тестов

Фигура	Позитивные	Отрицательные числа	Строки
Квадрат	4 из 4	0 из 2	0 из 2
Прямоугольник	6 из 6	0 из 2	0 из 2
Треугольник	6 из 6	0 из 2	1 из 2
Круг	4 из 4	0 из 2	2 из 2
Итого	**20 из 20**	**0 из 8**	**3 из 8**

8. Выводы из результатов тестирования

Все 20 позитивных тестов прошли успешно: площадь и периметр всех четырёх фигур вычисляются правильно, в том числе на нулевых, единичных и очень больших значениях.

Негативные тесты показали, что функции не проверяют входные данные. Отрицательные числа принимаются без ошибки, и функции возвращают бессмысленный результат, например отрицательный периметр. Строки вызывают TypeError только тогда, когда операцию не может выполнить сам Python (круг, площадь треугольника с одной строкой). В остальных случаях функции возвращают строку: например, периметр квадрата для 'abc' равен 'abcabcabcabc'.

Таким образом, расчёты выполняются верно, но функции не соответствуют требованию документации принимать только целые положительные числа, поэтому критерии приемки выполнены не полностью.

В ходе работы я научился писать позитивные и негативные unit-тесты с помощью unittest (в том числе с assertRaises), составлять план тестирования и подбирать тестовые случаи, а также разобрался, почему вещественные числа нужно сравнивать с учётом погрешности вычислений.